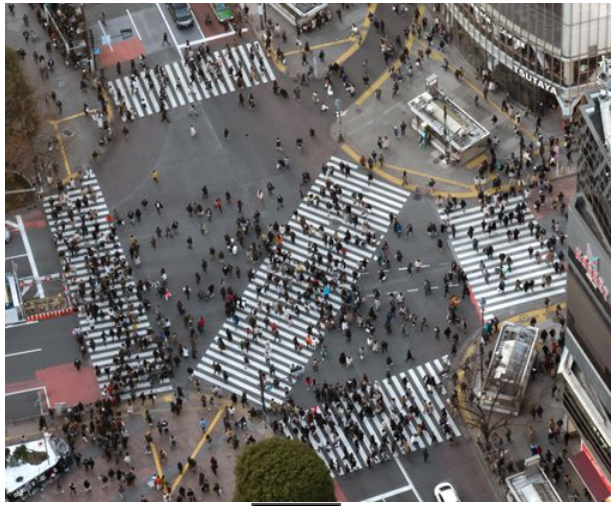
Route Finding App

Dropped as a Table

Our Problem

- People would often rather take a slightly longer route than have to walk on a route with lots of crossings or a lack of greenery etc.
- However, mainstream route finding apps only give users the most time efficient route and do not take into account other indicators
- There is a need for an app, specifically for walkers that allows users to set their preferences and generate routes based on those preferences



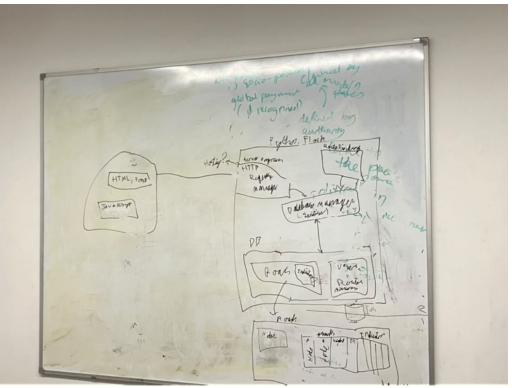
Approach and Design

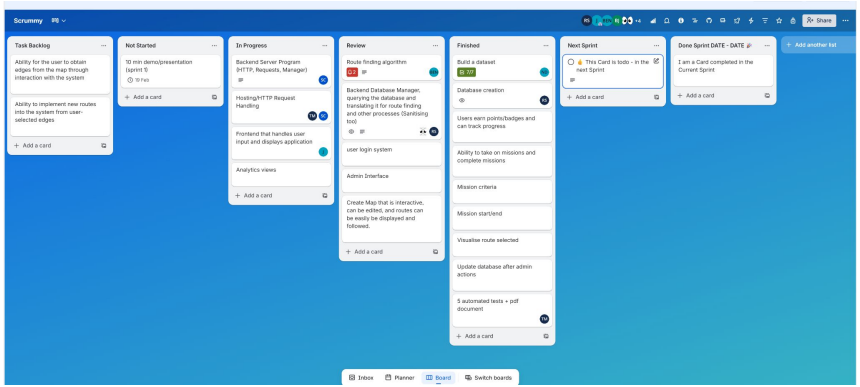
For this project, we took an Agile approach, where we had weekly sprints and scrum meetings.

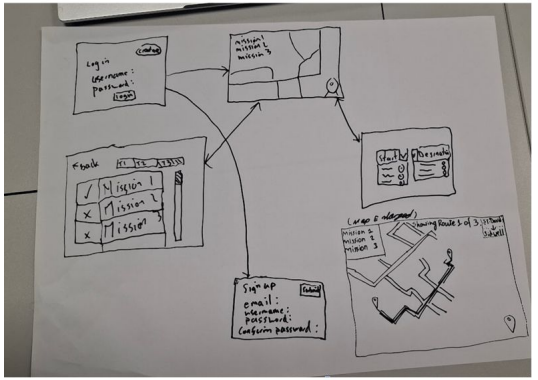
We broke down the project into smaller tasks (displayed on the scrum board pictured below) and during the scrum meetings we updated each other on progress and took on new tasks for the next week long sprint.

We all also took on a role which we were responsible for (i.e. Databases and SQL, Testing, Programming).

This was also where we decided the overall structure of the project and how the various states (map screen, mission select, etc…) would lead into each other.







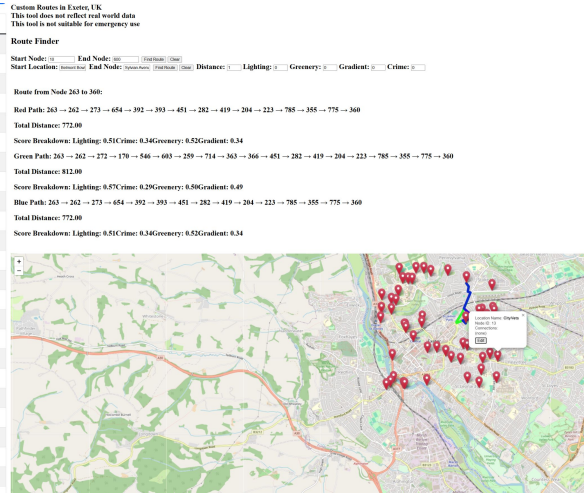
Implementation

Created a database From Openstreetmap Data and managed it with SQLite.

Used Flask, HTML, CSS, and JavaScript to create the web app.

Used SciPy for the route finding algorithm between locations.

nodeID	coordinatesX	coordinatesY	lighting	crime	greenery	gradient
0	-3.511056	50.722296	0.37	0.8	0.73	0.6
1	-3.535938	50.718241	0.16	0.01	0.06	0.87
2	-3.536843	50.71763	0.6	0.56	0.02	0.97
3	-3.537705	50.717846	0.83	0.06	0.18	0.18
4	-3.538113	50.718031	0.3	0.37	0.43	0.29
5	-3.536987	50.718869	0.61	0	0.29	0.37
6	-3.536861	50.719546	0.46	0.64	0.2	0.51
7	-3.535776	50.719558	0.59	0	0.61	0.17
8	-3.533024	50.719619	0.07	0.8	0.97	0.81
9	-3.529419	50.719905	0.3	0	0.68	0.44
10	-3.522206	50.725268	0.12	0.35	0.03	0.91
11	-3.523003	50.72532	0.26	0.51	0.31	0.52
12	-3.523353	50.725198	0.55	0.03	0.97	0.78
13	-3.508812	50.721817	0.94	0.74	0.6	0.92
14	-3.513665	50.722946	0.09	0.05	0.05	0.33
15	-3.516527	50.723639	0.39	0.12	0.83	0.36
16	-3.523108	50.727661	0.28	0.39	0.14	0.8
17	-3.53639	50.717694	0.07	0.84	0.77	0.2
18	-3.541471	50.719539	0.01	0.67	0.71	0.73
19	-3.538085	50.718403	0.77	0	0.36	0.12
20	-3.538697	50.718332	0.86	0.47	0.33	0.06
21	-3.541381	50.732525	0.31	0.18	0.73	0.64
22	-3.535913	50.727577	0.89	0.32	0.12	0.71



Demo

- We will show how the app finds multiple routes and how it made that decision.
- We will show that missions can be completed with feedback given to users.
- We will show that nodes can be added to the map by a trusted user.
- We will show that missions can be edited by trusted users.

Limitations

GUI and UX - ‘Prettify’ the main display page.

Execution time - Improve the route-finding speed.

Low variance in routes - Currently there is much overlap in routes.

Sprint 2

Requirements

- Have at least 6 viewable analytics (e.g. most popular destination) across the system.
- Audit log to track edits made.
- Role based access control (login, session management).
- Mobile-friendly layout.
- User feedback.

Risks

- Scope creep.
- Straying from project requirements.
- Problems from Sprint 1 flowing into Sprint 2.